

AW882XX Android Driver(MTK)

版本: V2.2

时间: 2023 年 11 月

目录

1. 驱动说明	3
2. 驱动移植	3
2.1 SMARTPA 配置	3
2.2 AW882XX 驱动移植	4
2.3 平台驱动配置	7
2.4 平台通路配置(仅 5G 平台需要配置)	9
2.5 驱动移植有效性验证	9
3. 算法集成	10
3.1 算法认证功能	10
4. IV 反馈功能验证	10
5. 校准功能	11
5.1 校准目的	11
5.2 校准适配	11
5.3 校准方式	11
5.4 校准有效性验证	14
5.5 校准示例代码	15
6. 调试接口	15
6.1 设备节点	15
6.2 KCONTROL	17
7. 附录	18
7.1 关于校准	18
7.2 平台 I2C 总线动态变更	19

1. 驱动说明

驱动源文件	aw882xx_calib.c, aw882xx_calib.h, aw882xx_monitor.c, aw882xx_monitor.h, aw882xx.c, aw882xx.h, aw882xx_device.c, aw882xx_device.h, aw882xx_dsp.c, aw882xx_dsp.h, aw882xx_init.c, aw882xx_log.h, aw882xx_spin.c, aw882xx_spin.h, aw882xx_data_type.h, aw882xx_pid_1852_reg.h, aw882xx_bin_parse.c, aw882xx_bin_parse.h, aw882xx_pid_2013_reg.h, aw882xx_pid_2032_reg.h, aw882xx_pid_2055_reg.h, aw882xx_pid_2055a_reg.h, aw882xx_pid_2071_reg.h, aw882xx_pid_2113_reg.h, aw882xx_pid_2308_reg.h
支持的产品（可以校准）	aw88257、aw88258、aw88261、aw88261s、aw88262、aw88263、aw88263h、aw88263s、aw88264、aw88265、aw88266、aw88266s、aw88270、aw88274、aw88299、aw88461
支持的产品（不可以校准）	aw88298、aw88298g、aw88266a、aw88230（无校准功能，请忽略 4/5 章的集成）
I ² C 地址范围	0x30/0x31/0x32/0x33/0x34/0x35/0x36/0x37
平台信息	mt6853

2. 驱动移植

2.1 SmartPA 配置

2.1.1 添加 PA 配置

在 ProjectXXX.mk 中添加.

```
#add aw882xx smartpa in ProjectXXX.mk
MTK_AUDIO_SPEAKER_PATH = smartpa awinic aw882xx
```

配置以上选项会在整编时在 AudioParamOptions.xml 中生成以下参数

```
<Param name="MTK_AUDIO_SPEAKER_PATH" value="smartpa_awinic_aw882xx" />
```

SmartPa_AudioParam.xml 添加 aw882xx 参数配置

```
--- a/audio_param/SmartPa_AudioParam.xml
+++ b/audio_param/SmartPa_AudioParam.xml
@@ -8,6 +8,7 @@
    <Param path="smartpa_cirrus_cs35l35" param_id="4"/>
    <Param path="smartpa_mtk_dummy" param_id="5"/>
    <Param path="smartpa_mtk_mt6660" param_id="5"/>
+   <Param path="smartpa_awinic_aw882xx" param_id="20"/>
  </ParamTree>
  <ParamUnitPool>
    <ParamUnit param_id="0">
@@ -76,5 +77,16 @@
    <Param name="is_apll_needed" value="1"/>
    <Param name="i2s_set_stage" value="4"/>
```

```

</ParamUnit>
+   <ParamUnit param_id="20">
+       <Param name="have_dsp" value="2"/>
+       <Param name="is_alsa_codec" value="1"/>
+       <Param name="chip_delay_us" value="22000"/>
+       <Param name="supported_rate_list"
value="0x1F40,0x2B11,0x2EE0,0x3E80,0x5622,0x5DC0,0x7D00,0xAC44,0xBB80"/>
+       <Param name="spk_lib_path" value=""/>
+       <Param name="spk_lib64_path" value=""/>
+       <Param name="codec_ctl_name" value=""/>
+       <Param name="is_apll_needed" value="1"/>
+       <Param name="i2s_set_stage" value="5"/>
+   </ParamUnit>
</ParamUnitPool>
</AudioParam>

```

SmartPa_ParamUnitDesc.xml 中添加描述

```
<Category name="smartpa awinic aw882xx"/>
```

2.2 AW882XX 驱动移植

2.2.1 DTS 配置

单 PA 配置方法

```

diff --git a/arch/arm/boot/dts/mediatek/mt6853.dtsi
b/arch/arm/boot/dts/mediatek/mt6853.dtsi
index f22db2e..a340a32 100644
--- a/arch/arm/boot/dts/mediatek/mt6853.dtsi
+++ b/arch/arm/boot/dts/mediatek/mt6853.dtsi
@@ -549,6 +549,8 @@
     i2c_x {
                                     /*x 表示对应的总线号*/
+
+       /* AWINIC AW882XX mono Smart PA */
+       aw882xx_smartpa_0: aw882xx_smartpa@34 {
+           compatible = "awinic,aw882xx_smartpa";
+           #sound-dai-cells = <0>;
+           reg = <0x34>;
+           reset-gpio = <&pio 89 0>; /*aw88230,aw88257,aw88261,aw88265 不能配置*/
+           irq-gpio = <&pio 37 0x0>;
+           /*sync-load = <1>;*/ /*固件加载方式,若驱动以 KO 方式编译时,需配置为 1*/
+           aw-re-min = <4000>; /*Re 校准范围最小值 (mOhms) */
+           aw-re-max = <30000>; /*Re 校准范围最大值 (mOhms) */
+           /*aw-cali-mode = "none";*/
+           status = "okay";
+       };
+       /* AWINIC AW882XX mono Smart PA End */

```

多 PA 配置方法

```

diff --git a/arch/arm/boot/dts/mediatek/mt6853.dtsi
b/arch/arm/boot/dts/mediatek/mt6853.dtsi
index f22db2e..a340a32 100644

```

```

--- a/arch/arm/boot/dts/mediatek/mt6853.dtsi
+++ b/arch/arm/boot/dts/mediatek/mt6853.dtsi
@@ -549,6 +549,8 @@
&i2c_x {
+                               /*x 表示对应的总线号*/
+   aw882xx_smartpa_0: aw882xx@34 {
+       compatible = "awinic,aw882xx_smartpa";
+       #sound-dai-cells = <0>;
+       reg = <0x34>;
+       reset-gpio = <&pio 89 0x0>;
+       irq-gpio = <&pio 37 0x0>;
+       /*sync-load = <1>;*/
+       sound-channel = <0>; /*0:pri_l 1:pri_r 2:sec_l 3:sec_r*/
+       aw-re-min = <4000>;
+       aw-re-max= <30000>;
+       /*aw-cali-mode = "none";*/
+       status = "okay";
+   };
+   aw882xx_smartpa_1: aw882xx@35 {
+       compatible = "awinic, aw882xx_smartpa";
+       #sound-dai-cells = <0>;
+       reg = <0x35>;
+       reset-gpio = <&pio 17 0x0>;
+       irq-gpio = <&pio 19 0x0>;
+       /*sync-load = <1>;*/
+       sound-channel = <1>; /*0:pri_l 1:pri_r 2:sec_l 3:sec_r*/
+       aw-re-min = <4000>;
+       aw-re-max= <30000>;
+       /*aw-cali-mode = "none";*/
+       status = "okay";
+   };
+};

```

2.2.2 驱动配置

defconfig 编译配置选项

```

#add aw882xx smartpa
CONFIG_SND_SMARTPA_AW882XX = y

```

在内核 **codecs** 目录下创建 **aw882xx** 目录，添加驱动文件

（如果如果 MTK 平台不带 DSP，则需要删除 **aw882xx_dsp.h** 中关于宏

AW_MTK_PLATFORM_WITH_DSP 的定义）

```

aw882xx_calib.c, aw882xx_calib.h, aw882xx_monitor.c,
aw882xx_monitor.h, aw882xx.c, aw882xx.h, aw882xx_device.c,
aw882xx_device.h, aw882xx_dsp.c, aw882xx_dsp.h, aw882xx_init.c,
aw882xx_log.h, aw882xx_spin.c, aw882xx_spin.h,
aw882xx_data_type.h, aw882xx_pid_1852_reg.h, aw882xx_bin_parse.c,
aw882xx_bin_parse.h, aw882xx_pid_2013_reg.h,
aw882xx_pid_2032_reg.h, aw882xx_pid_2055_reg.h,
aw882xx_pid_2055a_reg.h, aw882xx_pid_2071_reg.h,
aw882xx_pid_2113_reg.h, aw882xx_pid_2308_reg.h

```

Kconfig 配置

```
config SND_SMARTPA_AW882XX
    tristate "SoC Audio for awinic aw882xxseries"
    depends on I2C
    help
        This option enables support for aw882xxseries Smart PA.
```

Makefile 配置

```
#for AWINIC AW882XX Smart PA
obj-$(CONFIG_SND_SMARTPA_AW882XX) += aw882xx/aw882xx.o
aw882xx/aw882xx_monitor.o aw882xx/aw882xx_init.o
aw882xx/aw882xx_dsp.o aw882xx/aw882xx_device.o
aw882xx/aw882xx_calib.o aw882xx/aw882xx_bin_parse.o
aw882xx/aw882xx_spin.o
```

2.2.3BIN 文件配置

PA 需要配置寄存器等参数才能正常工作，PA bin 文件配置步骤如下：

编译配置

在项目对应位置添加 bin 文件编译选项

```
PRODUCT_COPY_FILES += \
xxxx/aw882xx_acf.bin:$(TARGET_COPY_OUT_VENDOR)/firmware/aw882xx_acf.bin

/*xxxx 为平台路径*/
```

路径配置

在内核 firmware_class.c 中添加 bin 文件在手机中的目录，一般目录为 **vendor/firmware**

```
static const char * const fw_path[] = {
    fw_path_para,
    "/vendor/firmware",          /*添加路径*/
    "/system/etc/firmware",
    "/lib/firmware/updates/" UTS_RELEASE,
    "/lib/firmware/updates",
    "/lib/firmware/" UTS_RELEASE,
    "/lib/firmware"
};
```

(PS: 调试阶段可直接 push bin 文件到/vendor/firmware/。)

bin 文件选择

请根据平台信号输出格式，PA 数量以及产品名称在移植包 **config** 目录下选择 bin 文件。

2.3 平台驱动配置

2.3.1 DAI_LINK 配置

不同平台下 dai_link 配置如下:

4G 平台

1) 添加 awinic_codecs 数组

单 PA 配置方法

```
struct snd_soc_dai_link_component awinic_codecs[] = {
    {
        .of_node = NULL,
        .dai_name = "aw882xx-aif-6-34",          /*以 I2C 总线 0x6, 地址 0x34 举例*/
        .name = "aw882xx_smartpa.6-0034",
    },
};
```

多 PA 配置方法

```
struct snd_soc_dai_link_component awinic_codecs[] = {
    {
        .of_node = NULL,
        .dai_name = "aw882xx-aif-6-34",          /*以 I2C 总线 0x6, 地址 0x34 举例*/
        .name = "aw882xx_smartpa.6-0034",
    },
    {
        .of_node = NULL,
        .dai_name = "aw882xx-aif-6-35",          /*以 I2C 总线 0x6, 地址 0x35 举例*/
        .name = "aw882xx_smartpa.6-0035",
    },
};
/*以上以双 PA 配置举例, 多 PA 时依次增加配置*/
```

2) 修改 DAI 接口

```
static struct snd_soc_dai_link mt_soc_extspk_dai[] = {
    {
        .name = "Ext_Speaker_Multimedia",
        .stream_name = MT_SOC_SPEAKER_STREAM_NAME,
        .cpu_dai_name = "snd-soc-dummy-dai",
        .platform_name = "snd-soc-dummy",
        + #ifdef CONFIG_SND_SMARTPA_AW882XX
        +   .num_codecs = ARRAY_SIZE(awinic_codecs),
        +   .codecs = awinic_codecs,
        + #endif
        .ops = &mt_machine_audio_ops,
    }
};
```

5G 平台

kernel 5.4 以下版本配置

单 PA 配置方法

```
diff --git a/arch/arm/boot/dts/mediatek/mt6853.dtsi
b/arch/arm/boot/dts/mediatek/mt6853.dtsi
index f22db2e..a340a32 100644
--- a/arch/arm64/boot/dts/mediatek/mt6853.dts
+++ b/arch/arm/boot/dts/mediatek/mt6853.dts
@@ -2824,7 +2824,7 @@
     mtk_spk_i2s_in = <0>;
     /* mtk_spk_i2s_mck = <3>; */
     mediatek,speaker-codec {
-        sound-dai = <&speaker_amp>;
+        sound-dai = <&aw882xx_smartpa_0>;
     };
};
/*根据 dts 配置 I2C 节点<aw882xx_smartpa_0>添加 dai 信息*/
```

多 PA 配置方法

```
diff --git a/arch/arm/boot/dts/mediatek/mt6853.dtsi
b/arch/arm/boot/dts/mediatek/mt6853.dtsi
index f22db2e..a340a32 100644
--- a/arch/arm64/boot/dts/mediatek/mt6853.dts
+++ b/arch/arm/boot/dts/mediatek/mt6853.dts
@@ -2824,7 +2824,7 @@
     mtk_spk_i2s_in = <0>;
     /* mtk_spk_i2s_mck = <3>; */
     mediatek,speaker-codec {
-        sound-dai = <&speaker_amp>;
+        sound-dai = <&aw882xx_smartpa_0 &aw882xx_smartpa_1>;
     };
};
/*以双 PA 举例，其中对应 I2C 节点的信息分别为<aw882xx_smartpa_0>、<aw882xx_smartpa_1>*/
```

kernel 5.4 及以上版本配置

单 PA 配置方法

```
/* 参考的修改文件: kernel_path/sound/soc/mt6789/mt6789_mt6366.c */
/* i2c0 */
SND_SOC_DAILINK_DEFS(i2s0,
    DAILINK_COMP_ARRAY(COMP_CPU("I2S0")),
    /*I2C 总线为 6，地址为 0x34 举例*/
    DAILINK_COMP_ARRAY(COMP_CODEC("aw882xx_smartpa.6-0034", "aw882xx-aif-6-34")),
    DAILINK_COMP_ARRAY(COMP_EMPTY()));
/* i2s3 */
SND_SOC_DAILINK_DEFS(i2s3,
    DAILINK_COMP_ARRAY(COMP_CPU("I2S3")),
    /*I2C 总线为 6，地址为 0x34 举例*/
    DAILINK_COMP_ARRAY(COMP_CODEC("aw882xx_smartpa.6-0034", "aw882xx-aif-6-34")),
    DAILINK_COMP_ARRAY(COMP_EMPTY()));
```


多 PA 配置方法

```
/* 参考的修改文件:kernel_path/sound/soc/mt6789/mt6789_mt6366.c */
/* i2c0 */
SND_SOC_DAILINK_DEFS(i2s0,
    DAILINK_COMP_ARRAY(COMP_CPU("I2S0")),
    /*I2C 总线为 6, 地址为 0x34 和 0x35 举例*/
    DAILINK_COMP_ARRAY(COMP_CODEC("aw882xx_smartpa.6-0034", "aw882xx-aif-6-34"),
        COMP_CODEC("aw882xx_smartpa.6-0035", "aw882xx-aif-6-35")),
    DAILINK_COMP_ARRAY(COMP_EMPTY()));
/* i2s3 */
SND_SOC_DAILINK_DEFS(i2s3,
    DAILINK_COMP_ARRAY(COMP_CPU("I2S3")),
    /*I2C 总线为 6, 地址为 0x34 和 0x35 举例*/
    DAILINK_COMP_ARRAY(COMP_CODEC("aw882xx_smartpa.6-0034", "aw882xx-aif-6-34"),
        COMP_CODEC("aw882xx_smartpa.6-0035", "aw882xx-aif-6-35")),
    DAILINK_COMP_ARRAY(COMP_EMPTY()));
```

2.4 平台通路配置(仅 5G 平台需要配置)

```
diff --git a/arch/arm/boot/dts/mediatek/mt6853.dtsi
b/arch/arm/boot/dts/mediatek/mt6853.dtsi
index f22db2e..a340a32 100644
--- a/arch/arm/boot/dts/mediatek/mt6853.dtsi
+++ b/arch/arm/boot/dts/mediatek/mt6853.dtsi
    sound: sound {
        compatible = "mediatek,mt6853-mt6359-sound";
        mediatek,audio-codec = <mt6359_snd>;
        mediatek,platform = <afe>;
+       mtk_spk_i2s_out = <3>;
+       mtk_spk_i2s_in = <9>;
        mtk_spk_i2s_mck = <3>;
    };
```

2.5 驱动移植有效性验证

以上操作完成了 RX 驱动集成, 通过以下驱动 log 确认移植有效:

I2C 通信成功:

```
[Awinic][6-0034]aw882xx_dai_drv_append_suffix: dai name [aw882xx-aif-6-34]
[Awinic][6-0034]aw882xx_dai_drv_append_suffix: pstream_name name [Speaker_Playback-6-34]
[Awinic][6-0034]aw882xx_dai_drv_append_suffix: cstream_name name [Speaker_Capture-6-34]
[Awinic][6-0034]aw882xx_i2c_probe: dev_cnt 1
```

声卡注册成功:

```
[Awinic] [6-0034] aw882xx_codec_probe: enter
[Awinic] [6-0034] aw882xx_add_codec_controls: enter
[Awinic] [6-0034] aw882xx_request_firmware: load [aw882xx_acf.bin] , file size: [2016]
[Awinic] aw_dev_parse_check_acf_by_hdr: project name [A2113]
[Awinic] aw_dev_parse_check_acf_by_hdr: custom name [Awinic]
```

bin 文件加载成功:

```
[Awinic] [6-0034] aw_monitor_parse_vol_data_v_0_1_1: ==parse vol end ==
[Awinic] [6-0034] aw_dev_parse_skt_type: enter
[Awinic] [6-0034] aw_dev_parse_skt_type: get dsp data prof cnt is 0
[Awinic] [6-0034] aw_dev_parse_get_vaild_prof: get vaild profile:2
[Awinic] [6-0034] aw_dev_parse_acf: parse cfg success
[Awinic] [6-0034] aw_dev_soft_reset: soft reset done
[Awinic] [6-0034] aw_dev_reg_fw_update: amppd_st=0x0000
```

播放音乐, PA 发声:

```
[Awinic] [6-0034] aw_dev_set_intmask: done
[Awinic] [6-0034] aw_monitor_start: enter
[Awinic] [6-0034] aw_check_bop_status: enter
[Awinic] [6-0034] aw_check_bop_status: check done! bop status is 0
[Awinic] [6-0034] aw_device_start: done
[Awinic] [6-0034] aw882xx_start_pa: start success
```

3. 算法集成

请参考 awinic 提供的算法集成文档进行集成。

3.1 算法认证功能

PA 工作时, 若出现规律播放白噪的情况, 时序如下: 播放音源 10s-->播放 3s 白噪-->播放音源 10s-->播放 3s 白噪-->.....

以上现象说明 awinic 算法认证失败, 请使能驱动中对应功能重新编译。

```
diff --git a/aw882xx_dsp.h b/aw882xx_dsp.h
index 998fa55..bce2a75 100644
--- a/aw882xx_dsp.h
+++ b/aw882xx_dsp.h
@@ -19,7 +19,7 @@

-/*#define AW_ALGO_AUTH_DSP*/
+#define AW_ALGO_AUTH_DSP

/*factor form 12bit(4096) to 1000*/
#define AW_DSP_RE_TO_SHOW RE(re) (((re) * (1000)) >> (12))
```

4. IV 反馈功能验证

Dump ADSP pcm 数据, 请打开并确认 iv 数据是否正确。

Dataout 与 IV dump 文件命名规则如下图所示:

AudioDspStreamManager.0.725.5402.TaskPlayback_datain.pcm	PCM 文件	5,168 KB
AudioDspStreamManager.0.725.5402.TaskPlayback_dataout.pcm /*dataout数据*/	PCM 文件	5,168 KB
AudioDspStreamManager.0.725.5402.TaskPlayback_echoref.pcm	PCM 文件	0 KB
AudioDspStreamManager.0.725.5402.TaskPlayback_ivdump.pcm /*iv数据*/	PCM 文件	5,168 KB

对比 dataout 数据和 iv 数据，如果播放内容一致，iv 数据响度会略小，说明 iv 反馈功能已生效。

5. 校准功能

5.1 校准目的

针对喇叭保护需求，AW882XX 驱动支持在产线对 speaker 进行校准，并将符合要求的 speaker 的 re 值写入到手机的 persist 分区中。在开机加载芯片配置时，驱动会将 persist 分区中读到的校准值写入保护算法中，达到喇叭保护的作用。

5.2 校准适配

5.2.1 校准文件保存路径适配

驱动（aw882xx_calib.c）中定义了保存校准值文件的路径如下：

```
#ifndef AW_CALI_STORE_EXAMPLE
/*write cali to persist file example*/
#define AWINIC_CALI_FILE "/mnt/vendor/persist/factory/audio/aw_cali.bin" /*保存校准值文件的路径*/
#define AW_INT_DEC_DIGIT 10
```

请确认手机中是否存在相同路径，无对应路径时会导致校准失败：

```
msm8996:/ # cd mnt/vendor/persist/factory/audio/
msm8996:/mnt/vendor/persist/factory/audio # pwd
/mnt/vendor/persist/factory/audio
msm8996:/mnt/vendor/persist/factory/audio # ls
aw_cali.bin
msm8996:/mnt/vendor/persist/factory/audio #
```

5.3 校准方式

awinic 提供了三种校准的方式，分别是 misc、class 和 attr 三种。

5.3.1 class 方式

1) 节点功能

节点	功能
/sys/class/smarterpa/cali_time	1.可配置校准 re 的延时时间 2.读取当前校准 re 的延时时间
/sys/class/smarterpa/re25_calib	1.单独开启 re 校准

	2. 设置 re 值
/sys/class/smarta/f0_calib	单独开启 f0 校准
/sys/class/smarta/re_show	校准结束后获取 Re 值
/sys/class/smarta/f0_show	校准结束后获取 f0 值
/sys/class/smarta/re_range	获取校准 re 值有效范围

2) 校准步骤

- 播放静音文件;
- 校准 re:

```
msm8996:/sys/class/smarta #
msm8996:/sys/class/smarta # cat re25_calib
pri_l:6428 m0hms pri_r:6280 m0hms
msm8996:/sys/class/smarta #
```

- 校准 f0:

```
msm8996:/sys/class/smarta #
msm8996:/sys/class/smarta # cat f0_calib
pri_l:832 pri_r:980
msm8996:/sys/class/smarta #
```

5.3.2misc 方式

AW882XX 驱动 misc 校准通过可执行文件来实现。按照以下步骤进行:

1) 获取可执行文件

```
AW882XX_Driver_MTK_v1.10.0 /*移植包根目录 (以V1.10.0为例) */
├── ap
│   └── smartpa_cali
│       ├── aw882xx_cali_32 /*32位系统校准可执行文件*/
│       └── aw882xx_cali_64 /*64位系统校准可执行文件*/
└── example_source_code
    ├── aw882xx_cali_attr_multi_mode.c
    ├── aw882xx_cali_attr_single_dev_mode.c
    └── aw882xx_cali_class_example.c
```

2) 配置可执行文件

```
C:\Users\AW882XX_Driver_MTK_v1.10.0\ap\smartpa_cali>adb push aw882xx_cali_32 /system/bin/
aw882xx_cali_32: 1 file pushed. 0.6 MB/s (34504 bytes in 0.052s)
C:\Users\AW882XX_Driver_MTK_v1.10.0\ap\smartpa_cali>adb shell chmod 0777 /system/bin/aw882xx_cali_32
```

3) 指令介绍

```
msm8996:/ # aw882xx_cali_32 /*执行命令*/
Calibration executables version: v0.3.10 /*文件版本号*/

./aw882xx_cali [dev_name] cmd [optional params] /*命令格式*/

./aw882xx_cali [dev_name] cali [cali_re_time(ms)] /*校准re、f0*/

./aw882xx_cali [dev_name] cali_re [cali_re_time(ms)] /*校准re*/

./aw882xx_cali [dev_name] cali_f0 [noise] /*校准f0*/

./aw882xx_cali [dev_name] get_spkr_status /*获取实时re、te、f0*/

./aw882xx_cali [dev_name] get_spkr_st /*获取实时re、te*/

./aw882xx_cali [dev_name] set_cali_re re_value1 [re_value2] /*设置校准re值*/

./aw882xx_cali [dev_name] cali_f0_q [noise] /*校准f0、q*/

./aw882xx_cali [dev_name] cali_all [cali_re_time(ms)] /*校准re、f0、q*/

./aw882xx_cali [dev_name] get_re_range /*获取校准re值有效范围*/

./aw882xx_cali dev_name set_params params_file /*设置算法参数*/
```

参数解释（注：[]代表该选项可不填）

dev_name	用于校准单个设备，不同设备所用的 dev_name 与其在 dts 中配置的 sound-channel 相对应，其对应关系如下： 0: aw882xx_smartpa_l 1: aw882xx_smartpa_r 2: aw882xx_smartpa_sec_l 3: aw882xx_smartpa_sec_r 不填写时默认校准所有设备。
noise	填写 noise 参数用于校准 f0 时播放白噪； 若客户选择自行播放白噪，则无需填写该参数。
cali_re_time	用于设置校准 re 值的时间，单位（ms），最少不得小于 1000ms； 不填写时将会使用默认校准时间 3000ms。
re_value1	需要设置到系统的校准电阻值（单位：mohm）。
params_file	算法参数对应的路径。

4) 校准步骤

- 播放静音文件；
- 启动校准：

```
msm8996:/ # aw882xx_cali_32 start_cali
[aw882xx_smartpa_l]cali_RE = 6429
[aw882xx_smartpa_r]cali_RE = 6346
[aw882xx_smartpa_l]cali_f0 = 836
[aw882xx_smartpa_r]cali_f0 = 979
```

5.3.3attr 方式

1) 节点路径

以 I2C 总线为 0x6，地址为 0x34 为例：

```
msm8996:/sys/bus/i2c/drivers/aw882xx_smartpa/6-0034 # /*节点路径*/
msm8996:/sys/bus/i2c/drivers/aw882xx_smartpa/6-0034 # ls
algo_ver cali_f0 dbg_prof dsp_re modalias name print_dbg reg uevent
awrw cali_re driver f0_show monitor phase_sync re_range rw
cali cali_time drv_ver fade_step monitor_update power re_show subsystem
```

2) 节点功能

节点	功能
cali_time	1.可配置校准 re 的延时时间； 2.读取当前校准 re 的延时时间。
cali	开启 re 和 f0 校准。
cali_re	1.单独开启 re 校准； 2.设置 re 值。
cali_f0	单独开启 f0 校准；
re_show	校准结束后获取 Re 值。
f0_show	校准结束后获取 f0 值。
re_range	获取校准 re 值有效范围

3) 校准步骤

a. 播放静音文件；

b. 校准 re:

```
msm8996:/sys/bus/i2c/drivers/aw882xx_smartpa/6-0034 #
msm8996:/sys/bus/i2c/drivers/aw882xx_smartpa/6-0034 # cat cali_re
pri_1:6371 mOhms pri_r:6380 mOhms
msm8996:/sys/bus/i2c/drivers/aw882xx_smartpa/6-0034 #
```

c. 校准 f0:

```
msm8996:/sys/bus/i2c/drivers/aw882xx_smartpa/6-0034 #
msm8996:/sys/bus/i2c/drivers/aw882xx_smartpa/6-0034 # cat cali_f0
pri_1:830 pri_r:980
msm8996:/sys/bus/i2c/drivers/aw882xx_smartpa/6-0034 #
```

5.4 校准有效性验证

1) 多次校准，确认校准 re 是否在有效范围内，且值不恒定(re 有效范围与硬件同事确认)，以 attr 方式校准两次为例：

```
msm8996:/sys/bus/i2c/drivers/aw882xx_smartpa/6-0034 # cat cali_re
pri_1:6909 mOhms pri_r:6796 mOhms
msm8996:/sys/bus/i2c/drivers/aw882xx_smartpa/6-0034 # cat cali_re
pri_1:6855 mOhms pri_r:6802 mOhms
msm8996:/sys/bus/i2c/drivers/aw882xx_smartpa/6-0034 #
```

2) 查看 re 值是否写入文件中：


```
msm8996:/ # cat mnt/vendor/persist/factory/audio/aw_cali.bin
6855      6802msm8996:/ #
msm8996:/ #
```

3) 播放音乐状态下，查看 dsp_re 节点，确认 dsp_re 节点 re 值与上述校准值相同：

```
msm8996:/ # cat sys/bus/i2c/drivers/aw882xx_smartpa/6-0034/dsp_re
6854      /*从dsp中读取re值时需要经过定点化运算，故存在误差，误差值为1*/
msm8996:/ # cat sys/bus/i2c/drivers/aw882xx_smartpa/6-0037/dsp_re
6801      /*从dsp中读取re值时需要经过定点化运算，故存在误差，误差值为1*/
msm8996:/ #
```

4) 重启手机，播放音乐，再次查看 dsp_re 节点，确认 dsp_re 节点中的值与文件中的 re 值相同。

5.5 校准示例代码

```
AW882XX_Driver_MTK_v1.10.0      /*驱动移植包根目录 (以V1.10.0为例) */
├── ap
│   ├── smartpa_cali
│   │   ├── aw882xx_cali_32
│   │   └── aw882xx_cali_64
│   └── example_source_code
│       ├── aw882xx_cali_attr_multi_mode.c /*attr多设备同时校准示例代码*/
│       ├── aw882xx_cali_attr_single_dev_mode.c /*attr单设备校准方式示例代码*/
│       └── aw882xx_cali_class_example.c /*class校准方式示例代码*/
```

6. 调试接口

6.1 设备节点

AW882XX Driver 创建多个设备节点可供调试，路径是 sys/bus/i2c/drivers/aw882xx_smartpa/*-00xx，其中*为 I2C bus number，xx 为 I2C address。

6.1.1 reg

节点名字	reg
功能描述	用于读写 aw882xx 的所有寄存器
使用方法	读寄存器值：cat reg 写寄存器值：echo reg_addr reg_data > reg （16 进制操作）
参考例程	cat reg （获取所有可读寄存器上的值） echo 0x04 0x0241 > reg （向 0x04 寄存器写值 0x0241）

6.1.2 rw

节点名字	rw
功能描述	用于读写 aw882xx 的单个寄存器
使用方法	读寄存器值: echo reg_addr > rw (16 进制操作) cat rw 写寄存器值: echo reg_addr reg_data > rw (16 进制操作)
参考例程	echo 0x04 > rw (读取 0x04 寄存器值) cat rw echo 0x04 0x0241 > rw (向 0x04 寄存器写值 0x0241)

6.1.3 driver_ver

节点名字	driver_ver
功能描述	用于获取驱动版本号
使用方法	获取版本号: cat driver_ver

6.1.4 dsp_re

节点名字	dsp_re
功能描述	用于设置或者获取算法中设定的 re 值
使用方法	获取 re: cat dsp_re 设置 re: echo 7000 > dsp_re

6.1.5 fade_step

节点名字	fade_step
功能描述	设置淡入淡出步进
使用方法	设置步进: echo step > fade_step 获取步进: cat fade_step
参考例程	echo 6 > fade_step (设置步进为 6) cat fade_step (获取当前淡入淡出步进)

6.1.6 dbg_prof

节点名字	dbg_prof
功能描述	场景切换 dbg 节点
使用方法	打开场景切换功能: echo 1 > dbg_prof 关闭场景切换功能: echo 0 > dbg_prof 获取节点状态: cat dbg_prof

6.1.7 phase_sync

节点名字	phase_sync
功能描述	相位同步功能开关节点
使用方法	打开相位同步功能: echo 1 > phase_sync 关闭相位同步功能: echo 0 > phase_sync 获取节点状态: cat phase_sync

6.1.8 print_dbg

节点名字	print_dbg
功能描述	i2c 写功能 dbg 打印节点
使用方法	i2c 写打印功能打开: echo 1 > print_dbg i2c 写打印功能关闭: echo 0 > print_dbg 获取节点状态: cat print_dbg

6.1.9 algo_ver

节点名字	algo_ver
功能描述	获取算法版本号
使用方法	获取算法版本号: cat algo_ver

6.1.10 monitor

节点名字	monitor
功能描述	用于开关 monitor 保护功能
使用方法	开启保护功能: echo 1 > monitor 关闭保护功能: echo 0 > monitor 保护模式获取: cat monitor

6.1.11 monitor_update

节点名字	monitor_update
功能描述	更新 monitor bin
使用方法	更新 monitor bin: echo 1 > monitor_update

6.2 Kcontrol

Kcontrol	功能	实例
----------	----	----

aw_dev_0_prof	模式选择	tinymix aw_dev_0_prof Music tinymix aw_dev_0_prof Receiver	选择 Music 模式 选择 Receiver 模式
aw_dev_0_switch	开关芯片	tinymix aw_dev_0_switch Enable tinymix aw_dev_0_switch Disable	开启芯片 关闭芯片
aw_dev_0_monitor	开关 monitor	tinymix aw_dev_0_monitor Enable tinymix aw_dev_0_monitor Disable	开启 monitor 关闭 monitor
aw882xx_rx_switch	开关 mec	tinymix aw882xx_rx_switch 0 tinymix aw882xx_rx_switch 1	关闭 mec 开启 mec
aw882xx_tx_switch	开关 tx	tinymix aw882xx_tx_switch 0 tinymix aw882xx_tx_switch 1	关闭 tx 开启 tx
aw882xx_spin_switch	设置旋转角度	tinymix aw882xx_spin_switch spin_90 tinymix aw882xx_spin_switch spin_180	旋转 90 度 旋转 180 度
aw882xx_fadein_us	设置淡入步进时间	tinymix aw882xx_fadein_us 500	设置淡入步进时间为 500
aw882xx_fadeout_us	设置淡出步进时间	tinymix aw882xx_fadeout_us 500	设置淡出步进时间为 500

7. 附录

7.1 关于校准

Awinic 提供了 misc ioctl 节点校准，device attr 节点校准以及 class 节点校准方式：

名称	说明
/dev/aw882xx_smartpa	支持多 PA 同时校准
/sys/class/smartpa	支持多 PA 同时校准
/sys/bus/i2c/drivers/aw882xx_smartpa/x-xx/	支持单个 PA 或者多个 PA 同时校准

代码默认 misc/class 多 PA 同时校准，device attr 节点既可以支持单个 PA 校准又可以支持多个 PA 校准；通过配置 aw882xx_calib.c 中的全局变量 is_single_cali 设置，FAE 或者客户可以根据需要设置：

```

19: #include <linux/slab.h>
20: #include <linux/fs.h>
21: #include <linux/miscdevice.h>
22: #include <linux/device.h>
23: #include <linux/kernel.h>
24: #include <linux/of.h>
25: #include <sound/core.h>
26: #include <sound/pcm.h>
27: #include <sound/pcm_params.h>
28: #include <sound/soc.h>
29:
30: #include "aw882xx.h"
31: #include "aw882xx_dsp.h"
32: #include "aw882xx_log.h"
33: #include "aw882xx_calib.h"
34:
35: static bool is_single_cali = false; /*if mutli dev cali false, single dev true*/
36:
37: static const char *cali_str[CALI_STR_MAX] = {"none", "start_cali", "cali_re",
38: "cali_f0", "store_re", "show_re", "show_r0", "show_cali_f0", "show_f0",
39: "show_te", "show_st", "dev_sel", "get_ver", "get_dev_num",

```

如果只有单个 PA，所有节点均是单 PA 校准。

7.2 平台 I2C 总线动态变更

若平台 I2C 总线号会发生变更，I2C 总线号的动态变更会导致 dai_link 匹配失败，可通过修改设备树配置与 dai_link 配置解决。

7.2.1 DTS 配置

驱动节点增加 rename-flag 属性，并配置属性值为 1。

单 PA 配置方法

```
diff --git a/arch/arm/boot/dts/mediatek/mt6853.dtsi
b/arch/arm/boot/dts/mediatek/mt6853.dtsi
index f22db2e..a340a32 100644
--- a/arch/arm/boot/dts/mediatek/mt6853.dtsi
+++ b/arch/arm/boot/dts/mediatek/mt6853.dtsi
@@ -549,6 +549,8 @@
     i2c_x {                                /*x 表示对应的总线号*/
+
+        /* AWINIC AW882XX mono Smart PA */
+        aw882xx_smartpa_0: aw882xx_smartpa@34 {
+            compatible = "awinic,aw882xx_smartpa";
+            #sound-dai-cells = <0>;
+            reg = <0x34>;
+            reset-gpio = <&pio 89 0>; /*aw88230,aw88257,aw88261,aw88265 不能配置*/
+            irq-gpio = <&pio 37 0x0>;
+            aw-re-min = <4000>;          /*Re 校准范围最小值 (mOhms) */
+            aw-re-max = <30000>;         /*Re 校准范围最大值 (mOhms) */
+            /*aw-cali-mode = "none";*/
+            rename-flag = <1>;
+            status = "okay";
+        };
+    /* AWINIC AW882XX mono Smart PA End */
```

多 PA 配置方法

```
diff --git a/arch/arm/boot/dts/mediatek/mt6853.dtsi
b/arch/arm/boot/dts/mediatek/mt6853.dtsi
index f22db2e..a340a32 100644
--- a/arch/arm/boot/dts/mediatek/mt6853.dtsi
+++ b/arch/arm/boot/dts/mediatek/mt6853.dtsi
@@ -549,6 +549,8 @@
    &i2c_x {                                /*x 表示对应的总线号*/
+
+        aw882xx_smartpa_0: aw882xx@34 {
+            compatible = "awinic,aw882xx_smartpa";
+            #sound-dai-cells = <0>;
+            reg = <0x34>;
+            reset-gpio = <&pio 89 0x0>;
+            irq-gpio = <&pio 37 0x0>;
+            sound-channel = <0>; /*0:pri_l 1:pri_r 2:sec_l 3:sec_r*/
+            aw-re-min = <4000>;
+            aw-re-max = <30000>;
+            /*aw-cali-mode = "none";*/
+            rename-flag = <1>;
+            status = "okay";
+        };
+    };
```

```
+   aw882xx_smartpa_1: aw882xx@35 {
+       compatible = "awinic, aw882xx_smartpa";
+       #sound-dai-cells = <0>;
+       reg = <0x35>;
+       reset-gpio = <&pio 17 0x0>;
+       irq-gpio = <&pio 19 0x0>;
+       sound-channel = <1>; /*0:pri_l 1:pri_r 2:sec_l 3:sec_r*/
+       aw-re-min = <4000>;
+       aw-re-max = <30000>;
+       /*aw-cali-mode = "none";*/
+       rename-flag = <1>;
+       status = "okay";
+   };
+};
```

7.2.2 DAI_LINK 配置

codec_name 与 codec_dai_name 修改后缀为 sound-channel，不同平台的 dai_link 配置区分如下：

4G 平台

3) 添加 awinic_codecs 数组

单 PA 配置方法

```
struct snd_soc_dai_link_component awinic_codecs[] = {
{
    .of_node = NULL,
    .dai_name = "aw882xx-aif-0", /*修改后缀为 dts 节点对应的 sound-channel */
    .name = "aw882xx_smartpa_0",
},
};
```

多 PA 配置方法

```
struct snd_soc_dai_link_component awinic_codecs[] = {
{
    .of_node = NULL,
    .dai_name = "aw882xx-aif-0", /*修改后缀为 dts 节点对应的 sound-channel */
    .name = "aw882xx_smartpa_0",
},
{
    .of_node = NULL,
    .dai_name = "aw882xx-aif-1", /*修改后缀为 dts 节点对应的 sound-channel */
    .name = "aw882xx_smartpa_1",
},
};
/*以上以双 PA 配置举例，多 PA 时依次增加配置*/
```

4) 修改 DAI 接口

```
static struct snd_soc_dai_link mt_soc_extspk_dai[] = {
{
    .name = "Ext_Speaker_Multimedia",
    .stream_name = MT_SOC_SPEAKER_STREAM_NAME,
    .cpu_dai_name = "snd-soc-dummy-dai",
}
```

```
.platform_name = "snd-soc-dummy",
#ifdef CONFIG_SND_SMARTPA_AW882XX
+ .num_codecs = ARRAY_SIZE(awinic_codecs),
+ .codecs = awinic_codecs,
#endif
.ops = &mt_machine_audio_ops,
}
}
```

5G 平台

5G 平台不需要做额外修改;